

advanced patterns

there are specialized patterns designed to handle dynamic work generation, speculative execution and irregular data structures

task and control-flow patterns

superscalar sequences

- this pattern involves writing a sequence of tasks where the execution order is constrained **only** by data dependencies
- the system automatically identifies tasks that are independent and executes them in parallel

il runtime costruisce un grafo delle dipendenze (DAG)
→ appena gli input di un task sono disponibili, questo viene eseguito, permettendo l'esecuzione parallela automatica di task indipendenti

similar to how superscalar processors handle instructions

futures

- this is similar to the **fork-join** pattern but offers more flexibility

key distinction

tasks do **not** need to be nested hierarchically

- a task can spawn a "future" (a promise of a result) and continue
- the result can be retrived later by a different task, breaking the strict parent-child structure of fork-join

speculative selection

- this is a generalized, parallel version of the serial **selection** (if/else) pattern
- mechanism**
 - the system executed the condition, the "then" branch and the "else" branch *all in parallel*
- outcome**
 - once the condition evaluates to true or false
 - the result from the correct branch is kept
 - the computation from the incorrect branch is discarded

the system executed the condition, the "then" branch and the "else" branch *all in parallel*

once the condition evaluates to true or false

- the result from the correct branch is kept
- the computation from the incorrect branch is discarded

this trades computational resources for reduced latency

nasconde la latenza della valutazione della condizione

workpile

- generalization of the **Map** pattern
- dynamic work generation**
 - unlike a standard Map where the number of iterations is fixed, in a Workpile, each instance of the elemental function can generate *more instances* of work
 - these new items are added to the "pile" of work to be processed, allowing the workload to grow dinamically

unlike a standard Map where the number of iterations is fixed, in a Workpile, each instance of the elemental function can generate *more instances* of work

mentre un task elabora un elemento, può generare nuovi elementi che vengono aggiunti alla "pila" (coda) condivisa → il pattern termina solo quando la pila è vuota e nessun task è attivo

these new items are added to the "pile" of work to be processed, allowing the workload to grow dinamically

common in graph traversals

data organization patterns

search

- filtrare i dati**
 - this pattern finds data within a collection that meets a specific set of criteria
- it typically involves a parallel traversal where multiple threads scan different parts of the collection

è una combinazione di map (per verificare il criterio su ogni elemento) con un meccanismo di uscita anticipata (abort) appena il dato viene trovato, interrompendo gli altri thread

potentially terminating early once the target is found

segmentation

- this involves operations on 1D collections that are subdivided into partitions

esegue operazioni su partizioni di una collezione 1D che sono suddivise, non sovrapposte e, soprattutto, di dimensioni non uniformi

segmentation

the partitions are **non**-overlapping and **non**-uniformly sized

this distinguishes it from standard blocking/tiling where partition sizeds are usually uniform

expand

- this pattern is defined as a combination of the **Pack** and **Map** patterns
- it is used when an input element needs to generate data that effectively expands the size of the collection

ex. decompression or generating vertices from a geometric definition

category reduction

- this is a specific data aggregation pattern
- input**
 - a collection of elements, where each element has an associated **label** (or key)
- operation**
 - the system finds all elements sharing the same label and **reduces** them

the system finds all elements sharing the same label and **reduces** them

conceptually similar to the "Reduce" phase in MapReduce

Futures:

- **Concetto:** Simile al Fork-Join, ma più flessibile perché non richiede una struttura gerarchica stretta.
- **Funzionamento:** Quando un task viene lanciato, ritorna immediatamente un oggetto "Future" (un segnaposto o promessa). Il chiamante continua l'esecuzione e si blocca solo quando prova a leggere il valore effettivo contenuto nel Future. Se il task non ha ancora finito, attende in quel momento .

1. #pragma omp for → Pattern Map

Hai ragione: quando usi `#pragma omp for`, stai implementando un pattern **Map**.

- **Perché:** Il pattern *Map* replica una funzione su ogni elemento di una collezione di dimensione nota .
- **In OpenMP:** La direttiva `#pragma omp for` serve proprio a questo: prende un ciclo con limiti definiti (la "collezione") e divide le iterazioni tra i thread del team. È l'implementazione classica per carichi di lavoro statici o prevedibili ("loop parallelism").

2. #pragma omp task → Pattern Workpile

Hai ragione anche qui: quando usi `#pragma omp task` per generare lavoro dinamicamente, stai spesso implementando un pattern **Workpile**.

- **Perché:** Il pattern *Workpile* è una generalizzazione della Map dove i task attivi possono generare **nuovi task** durante l'esecuzione, aggiungendoli a una "pila" (coda) di lavoro condivisa . È fondamentale quando la quantità totale di lavoro **non è nota a priori** (es. esplorazione di un grafo o di un albero).
- **In OpenMP:** La direttiva `#pragma omp task` definisce un'unità di lavoro che può essere eseguita subito o differita. Questo meccanismo permette a un thread, mentre sta eseguendo un pezzo di codice, di creare ("spawnare") nuovo lavoro dinamicamente senza dover sapere all'inizio quanti task ci saranno in totale.

3. Expand (Espansione)

- **Definizione:** È una combinazione di **Pack** e **Map** che permette a ogni singolo elemento di input di generare un numero variabile di elementi di output (0, 1 o molti).
- **Come funziona:**
 1. Si calcola quanti elementi di output genererà ciascun elemento di input (es. l'elemento A ne crea 3, B ne crea 0, C ne crea 2).
 2. Si usa uno **Scan** su questi conteggi per calcolare gli indici di scrittura nell'array di destinazione.
 3. Si scrivono i dati in parallelo.
- **Utilizzo:** È cruciale per la generazione dinamica di dati, come nella grafica computerizzata (tassellazione: un triangolo viene diviso in 4 triangoli più piccoli) o in algoritmi dove i dati "crescono" dinamicamente .